Unit 1 – Basics, Data Types, Templates, Functions

1. What is Go language?

Go (Golang) is an **open-source programming language** developed by Google. It is **fast, simple, and supports concurrency**.

---

2. What is the structure of a Go program?

Every Go program starts with:

- package main
- import
- main() function

---

3. What are variables in Go?

Variables store data values.

Example:

var a int = 10

b := 20

---

4. Difference between var and :=

var	**:=**
used anywhere	only inside function
need datatype sometimes	datatype auto detected

---

5. What are basic data types?

- int

- float
- string
- bool

---

6. What is an array?

Array is fixed size collection.

var a [3]int

---

7. What is slice?

Slice is dynamic size array.

s := []int{1,2,3}

---

8. Difference between array and slice

Array	Slice
fixed size	dynamic size
memory fixed	flexible

---

9. What is map?

Map stores key-value pairs.

m := map[string]int{"age":22}

---

10. What is struct?

Struct is user-defined data type.

```
type Student struct{
 name string
```

}

---

11. What is JSON?

JSON is data format used for API communication.

---

12. Difference between text template and HTML template

Text Template	HTML Template
plain text	web pages
no security	auto escaping

---

13. What is function?

Function is block of reusable code.

---

14. What is method?

Method is function attached to struct.

Unit 2 – Interfaces

15. What is interface in Go?

Interface defines method behavior.

---

16. What is interface satisfaction?

If struct implements methods, it automatically satisfies interface.

---

17. Does Go use implements keyword?

No, Go automatically checks methods.

---

18. What is empty interface?

Interface with no methods.

var a interface{}

---

19. What is type assertion?

Used to get actual value from interface.

value := i.(string)

---

20. What is type switch?

Switch used for multiple types.

---

21. What is error interface?

Error is built-in interface.

```
type error interface{
 Error() string
}
```

---

22. What is http.Handler interface?

Used in web servers to handle requests.

---

23. What is sort.Interface?

Used to sort collections.

---

24. What is flag package?

Used to read command line input.

Unit 3 – Goroutines & Concurrency

25. What is goroutine?

Goroutine is lightweight thread.

---

26. How to create goroutine?

Using go keyword.

go function()

---

27. What is channel?

Channel is used to communicate between goroutines.

---

28. Types of channels

- buffered
- unbuffered

---

29. What is select statement?

Select handles multiple channels.

---

30. What is race condition?

When multiple goroutines access same variable.

---

31. What is Mutex?

Mutex prevents race condition.

---

32. What is RWMutex?

Allows multiple readers, one writer.

---

33. What is sync.Once?

Executes code only once.

---

34. Difference between concurrency and parallelism

Concurrency	Parallelism
tasks managed	tasks executed simultaneously

---

35. What is race detector?

Tool to detect race condition.

go run -race main.go

Unit 4 – Packages and Testing

36. What is package?

Collection of Go files.

---

37. What is main package?

Executable program package.

---

38. What is import keyword?

Used to include packages.

---

39. What is blank import?

Import without using directly.

import _ "image/png"

---

40. What is go build?

Compiles program.

---

41. What is go run?

Runs program.

---

42. What is go test?

Runs test cases.

---

43. What is unit testing?

Testing small part of program.

---

44. What is benchmark testing?

Checks performance.

---

45. What is code coverage?

Shows tested code percentage.

Unit 5 – Reflection & Low Level Programming

46. What is reflection?

Reflection checks type at runtime.

---

47. What is reflect.Type?

Returns datatype.

---

48. What is reflect.Value?

Returns value.

---

49. Why reflection is used?

To write generic code.

---

50. What are struct tags?

Metadata for struct fields.

Example:

Name string `json:"name"`

---

51. What is unsafe package?

Used for low level memory operations.

---

52. What is unsafe.Sizeof?

Returns memory size.

---

53. What is unsafe.Alignof?

Returns memory alignment.

---

54. What is unsafe.Offsetof?

Returns struct field position.

---

55. What is unsafe.Pointer?

Used to convert pointer types.

---

1. What is goroutine?

A **goroutine** is a lightweight thread managed by Go runtime.
It allows functions to run **concurrently**.

go myFunction()

👉 Used for fast and parallel execution.

---

2. What is interface?

An **interface** defines a set of method signatures (behavior).
If a struct implements those methods, it automatically satisfies the interface.

```
type Shape interface{
 area() float64
}
```

---

3. Difference between slice and array

Array	Slice
Fixed size	Dynamic size
Length cannot change	Length can change
[3]int	[]int
Faster	More flexible

Example:

```
arr := [3]int{1,2,3}
slice := []int{1,2,3}
```

---

4. What is struct?

A **struct** is a user-defined data type used to group variables.

```
type Student struct{
```

```
 Name string
 Age int
}
```

Used to represent real-world objects.

---

5. What is channel?

A **channel** is used for communication between goroutines.

```
ch := make(chan int)
ch <- 10
```

Channels help share data safely.

---

6. What is mutex?

Mutex (Mutual Exclusion) prevents multiple goroutines from accessing the same variable at the same time.

```
var mu sync.Mutex


mu.Lock()
count++
mu.Unlock()
```

Used to avoid race conditions.

---

7. What is package?

A **package** is a collection of Go source files.

Example:

```
package main
```

Used for code organization and reusability.

---

8. What is reflection?

Reflection allows a program to inspect type and value at runtime.

reflect.TypeOf(x)

reflect.ValueOf(x)

Used in generic programming.

---

9. What is JSON?

JSON (JavaScript Object Notation) is a format used to store and exchange data.

Example:

```
{
 "name":"Utsav",
 "age":22
}
```

Used in APIs and web applications.

---

10. What is type assertion?

Type assertion is used to get the actual value from an interface.

```
var i interface{} = "Go"
s := i.(string)
```

Used when interface holds multiple types.